

COMP2054-ADE

Lec19

Map ADT using Hash Maps

Hash Maps / Tables

- These are a CDT useful for implementing the MAP ADT
- **Basic idea: convert each key into an index into a (big) array**
 - The array is called the “hash table”
 - The conversion of key to index is called a “hash function”
- Look-up of keys and insertion and deletion in a hash table usually runs in $O(1)$ time.
 - Not guaranteed, and design of the table needs to be done carefully if want the access to be “reliably $O(1)$ ”

Applications of Hashing

- Direct applications of hash tables:
 - small databases
 - compilers
 - browser caches – the weird and wonderful filenames in the browser cache folder are hashcodes of something?
- Hash tables as an auxiliary data structure in a program:
 - Look-up table: if you want to check whether some object has been seen before, for example in a graph or list traversal, keep a hashtable of (object, "seen before") pairs, where the key is the reference to the object, and the value is some arbitrary marker.

Hash Functions

- A hash table (or associative array) will typically allow the key to be an arbitrary type, and does not restrict it to be an integer.
 - E.g. it could be a string "cat" or "dog"
- Even if the key is an integer, it may well not be in a suitable range to be an index
 - E.g. it could be negative or too large -1 or 391507515
- We need a "hash function", h , to convert Key k into a suitable index $h(k)$ into the array. E.g.

`unsigned int h(Key k) { ... }`

Hash Function: convert and compress

- The hash function typically has two stages
 1. Conversion to a (large unsigned) int
 2. Compression of the range of integers to allow them to be used as an index in the hash table

unsigned int h(Key k) { ... }

Hash Function: Collisions

- It is standard with hash functions that there can be multiple keys that map to the same index
- If two keys have
$$h(k1) = h(k2)$$
this is called a “collision”
- In this case, we must decide how to handle the collision.
- This collision handling makes the hash map less efficient
- Hence, we want hash functions that reduce the chance of collisions
 - If we can reduce the collisions to zero then this is called a “perfect hash” – but it is rare, and then we are basically using the resulting key as if it were in a database

Hash Function: Reducing Collisions

- We want to reduce the chance of two keys mapping to the same index
 - Want to spread the value of $h(k)$ out uniformly
 - We want no discernable patterns
 - Otherwise, a similar pattern in the distribution of the keys could cause an “alignment” that increased the collisions
 - Example:

if $\text{isEven}(k)$ implies $\text{isEven}(h(k))$

If the keys all happen to be even, then only half the table would be used, and the collisions would increase

Hash Function: Checksums

- No properties of the k should be “easily predictable” given $h(k)$
- This is a bit like “cryptographic” as we are hiding information about the k .
- There are “cryptographic hash functions” that do this very well – they can be used as “checksums” e.g. MD5
- Checksums convert a (a document or a file) to a shorter string with it not practical to invert to give the document and it very unlikely that two documents give the same hash function
- The catch for using these is that they would be too slow to compute for general
- When hashing a large number of keys we need the hash function to be fast to compute

Hash Function: Birthday paradox

- Even when $h(k)$ values are (pseudo-)randomly distributed, then collisions are more likely than might be expected.

Example:

- k =person $h(k)$ =birthday (day and month, but not year)
- Birthday paradox: If we have a random group of people, then how many do we need to have a 50% chance that two (or more) of them have the same birthday?
 - Note: there are 365 possible birthdays and they are uniformly and randomly distributed
 - Bad answer: So maybe need to fill about 50% of potential birthdays
– i.e. maybe need around 180 people
- Actual answer: Just 24 people are needed (!)
- Collisions are more likely than might be intuitive
- Random numbers tend to cluster more than expected

Hash Function: “convert” [not assessed]

Need to convert key, of any type, into an integer.

Should work for arbitrary sizes, e.g. strings

There are many methods:

https://en.wikipedia.org/wiki/Hash_function#Hashing_variable-length_data

Some options:

1. For objects can use the memory address
2. Simply reinterpret the bits as an integer
3. For strings, break into short portions, treat each as an integer, and combine them using a polynomial

Hash Function: “compress”

Given a hash table of size N we need to convert the integer key, k , to be in the range $0, \dots, N-1$

Obvious choice:

$$h(k) = k \bmod N$$

But this tends to lead to too many collisions, so generalize to “Multiply, Add and Divide” (MAD)

$$h(k) = (a k + b) \bmod N$$

Where a and b are carefully-chosen constants

Hash Function: compress by MAD

“Multiply, Add and Divide” (MAD)

$$h(k) = (a k + b) \bmod N$$

Where a and b are carefully-chosen constants.

Suppose that we had $a=6$ and $N=16$ and b odd

Then all keys would map to odd entries of the hash table and so would only use half the table.

This is because 6 and 16 have a factor, 2, in common.

Hence, we need that a and N are co-prime – no factors in common.

E.g. if N is a power of 2, then a should not be even

Collisions: Separate Chaining

The simplest way to handle collisions is via separate chaining

At each index, when it is used, we create a pointer to a separate Map that handles all the keys hashing to that index

All the insert/delete operations are simply delegated to the (list-based) Map stored at $h(k)$

Since the number of keys should be much smaller, then it might well be practical to use a list-based Map

Separate Chaining: Example

- Suppose we were hashing students with the key being the 8-digit Student ID, SID
- If took Table of size $N=1000$ and the simple hash function $h(k) = \text{SID} \bmod 1000$ then we can end up with far too many collisions:

h(k)	Ptr	
...		
802	0	
803		→ 20737802 → 20747802 → 20637802
804	0	
805	0	
806	0	
807		→ 20597807
808	0	
...		

Collisions: Open addressing

- Separate chaining has the disadvantage of requiring extra memory.
- Instead, we try to find ways to use existing spaces in the existing hash table “whenever possible”
- This is only possible if the key k has the possibility to be placed at an index other than $h(k)$
- Such systems are called “Open Addressing”:
- We will consider two methods of “open addressing”:
 1. “Linear probing”
 2. “Double hashing”

Collisions: Linear Probing

Given a k key, if the entry for $h(k)$ is already occupied then we move to the right until we find an empty position.

The array is treated in a circular fashion – once we get to the end then we start again at the beginning

That is, we try (or “probe”)

$h(k)$

$(h(k) + 1) \bmod N$

$(h(k) + 2) \bmod N$

$(h(k) + 3) \bmod N$

...

Until

- A probe succeeds: we find an empty cell, and place the entry there
- We have tried all cells, and the hash table is full, in which case will need to expand the table (see later)

Linear Probing : Example

- Example: $N=8$ $h(k) = k \bmod 8$
- Insert keys 14, 15, 8, 22 in this order:
 - 14,15,8 have no collisions, simply giving

8						14	15
---	--	--	--	--	--	----	----

- But $22 \bmod 8 = 6$ collides with $14 \bmod 8 = 6$,
 - Probe index = $(6+1) \bmod 8 = 7$ - fails as occupied
 - Probe index = $(6+2) \bmod 8 = 0$ - fails as occupied – note the circular probe
 - Probe index = $(6+3) \bmod 8 = 1$ - now succeeds and we get

8	22					14	15
---	----	--	--	--	--	----	----

- Note this case had quite a long walk to find a space
 - The keys happened to have hash values towards the end of the range
 - Generally, would expect to just have “short” walks

Linear Probing : Example : get

- Example: $N=8$ $h(k) = k \bmod 8$
- When we lookup a key we need to follow the same procedure and walk right until
 - Find the key
 - Find an unoccupied space, and can then stop – if the key were present then the key would have been placed there
- E.g. in the previous example, we arrived at

8	22					14	15
---	----	--	--	--	--	----	----

- Now suppose we want to do `get(22)`
 - $h(22) = 6$ and so we start probing from `index=6`
 - In this case we do find the `key=22` and can return the value
- In contrast, if we were asked for `get(7)`
 - $h(7)=7$ and so we start probing from `index=7`
 - But we reach the empty cell at `index=2` before finding 7
 - Hence can conclude 7 was not stored (return 'key not present')

Linear Probing : Deletion

- Example: $N=8$ $h(k) = k \bmod 8$
- We arrived at

8	22					14	15
---	----	--	--	--	--	----	----

- But what if now want to remove(15)
- Naively we might just get

8	22					14	
---	----	--	--	--	--	----	--

- Do you see a problem?
- Suppose we try to do get(22)
- The probes would stop at the empty cell at index=7
- Would incorrectly fail to find the key=22 !!

Linear probing: Deletions

- How do you **safely** remove an element x?
- One answer:
 - (not the best answer, but simplest):
 - Find x using `get` and set the entry back to blank, i.e. null or empty (which sometimes write as `#`)
 - Fix the sequence on its right-hand-side
 - WHY!?: If any entry on the right used linear probing then it might no longer be discoverable by `get` because it will stop at the blank!!!
 - Fix: Move such entries, e.g. by removing them and then re-inserting them all.
 - **EXERCISE (offline)** Figure out the details and write pseudo-code for this and do examples. (See tutorials for examples)

Linear Probing: Lazy deletion

How do you safely remove an element x ?
Another solution:

- “**Lazy deletion**”: don’t mark the entry as a blank, but as a ‘deleted’ and fix the entries later. E.g. see

http://opendatastructures.org/versions/edition-0.1e/ods-java/5_2_LinearHashTable_Linear_.html

- E.g. in the find, skip over a ‘deleted’ entry rather than stopping

(See Tutorials)

Collisions: Linear Probing – big jumps

Instead of just moving one cell to the right, we could do a jump of d cells

That is, we try

$h(k)$

$(h(k) + d) \bmod N$

$(h(k) + 2d) \bmod N$

$(h(k) + 3d) \bmod N$

...

However, if we had $h(k)=1$, $d=4$ and $N=16$

This would just try 1,5,9,13

And then try $(13+4) \bmod 16=1$ which had already failed.

Again, this is because 4 and 16 have a factor, 4, in common.

So, we require that d and N are co-prime

Collisions: Double hashing

This is an extension of “Linear probing” but in which the size of the jump is allowed to depend on the key.

In particular, it depends on a “secondary hash function” $d(k)$

That is, we try

$h(k)$

$(h(k) + 1 * d(k)) \bmod N$

$(h(k) + 2 * d(k)) \bmod N$

$(h(k) + 3 * d(k)) \bmod N$

...

Obviously, we need $d(k) > 0$, otherwise we just probe the same cell again

Double hashing: Example

Take $h(k) = k \bmod 8$ again, but with
 $d(k) = 1 + (4 * k) \bmod 7$ - always in the range 1...7

Again, insert keys 14, 15, 8, 22.

Firstly, compute the h and d values:

k	h(k)	d(k)	(h(k)+d(k)) mod 8
14	6	1	
15	7	5	
8	0	5	
22	6	5	3

The 14,15,8 do not collide so again get

8						14	15
---	--	--	--	--	--	----	----

Then 22 collides with 14 as both have $h(k)=6$, so just probe $(h(22)+d(22)) \bmod 8$, which is free, so place it there:

8			22			14	15
---	--	--	----	--	--	----	----

The jumps are bigger, and different values, so less likely to have long walks of probing

Performance: Best case

- Suppose we have a Table size N , and with n entries
- Best case: $O(1)$
 - If we have no collisions, then the insert and add are simply a matter of looking at index $h(k)$
 - This is like access to an array and so is $O(1)$
 - Though we do have the extra cost of computing the hash function

Performance: Worst case

- Suppose we have a Table size N , and with n entries
- Worst-case: $O(n)$
 - As it might happen that all keys have the same hash value, and so
 - Separate chaining: has a list of length n
 - Linear Probing: we get a sequence on n entries and have to walk to n entries
 - Double hashing: here would need all keys to happen to have the same $h(k)$ and $d(k)$ and so is less likely – but again may lead to needing $O(n)$

Performance: “Typical” case

- Suppose we have a Table size N , and with n entries
- Typical or average case will often still be still $O(1)$, or close, as collisions are rare
 - Separate chaining: the individual lists are short
 - Open addressing: the sequences of entries are short we do not have to try many alternative addresses
- However, this relies on the table not being too full:

Performance: Load factor

- Suppose we have a Table size N , and with n entries
- The “load factor” is the fraction, or percentage, of entries that are occupied that is n/N
- When the load factor is “too large” then the performance will become much worse.
- At some load factor, e.g. 75%, then the system may decide to increase the size of the hash table, and do a “re-hash”

Re-Hashing

When the table gets too full then “re-hash”:
Create a new larger table and new hash function.

- Need to (eventually) transfer all the entries from the old table to the new one
- If do so immediately, then
 - one can amortise the cost over many entries (as for Vector) and so get an average cost of $O(1)$ again
 - but the worst case might be $O(n)$ when the table is rehashed, and this might be bad for a real time system
 - Option:
do not transfer all entries “in one go” but do “a few at a time”
 - Keep both tables until the transfer is complete; but only do insertions into the new table.

Exercise (offline): consider this in more detail, and read the relevant part of the text book (Section 9.2.7 Load factors and Reshaping) or the wiki page http://en.wikipedia.org/wiki/Hash_table#Dynamic_resizing

Comparison of HashMap and PQ

- HashMap does not use the ordering of keys
 - E.g. does not implement `min()`
 - In a hash table it would need a scan of all the keys in the table, so $O(n)$ (or worse)
- PQ does not allow direct access to a key
 - E.g. there is no easy way to do `get(k)`
 - In a (standard) heap we would have to walk through all the entries

Minimum Expectations

- Map ADT, and its usage
- basic concepts of hash tables
- hash codes and compression functions
- Options to handle collisions
 - Separate chaining, linear probing, double hashing
 - How to insert, find/get, remove for all these systems